Forecasting Project
Process Documentation
V2.0

Contents

Introduction	**3**
Data Extraction Process	**4**
Data Diagnostics	**5**
Target Variable	**11**
Predictive Variable	**11**
Pre-modeling	**12**
Models	**13**
Results	**18**
Future Work & Limitations	**21**
Agentic AI Application	**21**
Team Information	**24**

Introduction

This report details the development and implementation of a forecasting system designed to optimize electricity load management across the Portuguese power grid. As modernized grids face increasing volatility and changing consumption patterns, typical forecasting methods are insufficient for precise operational planning, leading to wasted costs. To address this issue, our team developed a cluster-optimized system that provides 24-hour predictions for 370 unique grid clients, utilizing SOTA and showcasing development from base models to see improvement.

By segmenting our client base into four distinct behavioral clusters, we were able to deploy specialized models that outperform legacy baselines like SARIMA or Prophet. These specialized models capture non-linear consumption “bases” and seasonal shifts that traditional methods often misinterpret as noise, resulting in significantly reduced error rates (MAPE) across both residential and high-demand industrial sectors.

Lastly, this solution is integrated into an Agentic AI framework, allowing for real-time, natural language querying of forecasts. This transition from static reporting to an automated system provides grid managers with the ability to reduce energy procurement costs, minimize costly reserves, and maintain long-term grid stability. The following sections detail our data engineering process, modeling strategy, and evaluation that confirms this system’s predictive performance.

Data Extraction Process

This section will detail the transformation process of taking the raw data into a structured format suitable for forecasting. To ensure model robustness and prevent information leakage, the extraction process followed a pipeline of cleaning, resampling, and temporal segmentation.

1. Dataset Overview and Cleaning

The raw dataset, sourced from the UCI Electricity Load Diagrams (2011-20134), contains 15-minute load readings for 370 unique clients (MT_001 to MT_370). Initial preprocessing involved:

- Scheme Standardization: Semicolon delimiters and comma-based decimal notation were converted to a standard CSV format to ensure compatibility with Python-based data science libraries.
- Temporal Fixes: Data from 2011 was excluded due to a significant volume of null values and "0s" representing inactive meters, which would have introduced artificial bias into the training phase.

2. Resampling and Noise Reduction

While the original data is recorded at 15-minute intervals, the data was resampled to an hourly frequency. This serves two critical business and technical purposes:

- Signal Stabilization: Reducing high-frequency noise and volatility that does not contribute to 24-hour trend analysis
- Operational Alignment: Aligning the model output with the standard day-ahead forecasting requirements. This resulted in a total of 26,304 hourly observations per client across the remaining three year span.

3. Normalization and Scaling

To account for the vast difference in scales, ranging from small residential units to large industrial facilities, we applied a StandardScaler. Each client's time series was transformed to have a mean of 0 and a standard deviation of 1. This ensures high-consumption clients do not dominate the loss function, allowing the models to learn the shape rather than the magnitude of load profiles.

4. Clustering

We implemented a cluster optimized architecture to ensure we are not treating all 370 clients as a single average or implementing 370 different problems.

- Segmentation: Clients were grouped into four distinct clusters using K-means clustering based on usage profiles (weekday/weekend patterns, etc)
- Windowing logic: For the deep-learning models, data was restructured into supervised input-output pairs using a 96-hour look-back window to predict a rolling 24-hour future.

5. Temporal Splitting

A chronological split was maintained to prevent data leakage:

- Training Set: 2012-2013 is used for model fitting and hyperparameter tuning
- Test Set: 2014 is reserved for out-of-sample evaluation.

Data Diagnostics

This section outlines the diagnostic procedures performed to understand the underlying structure of the Portuguese electricity grid data. These diagnostics were critical to justifying our move towards a cluster architecture rather than a global-average approach.

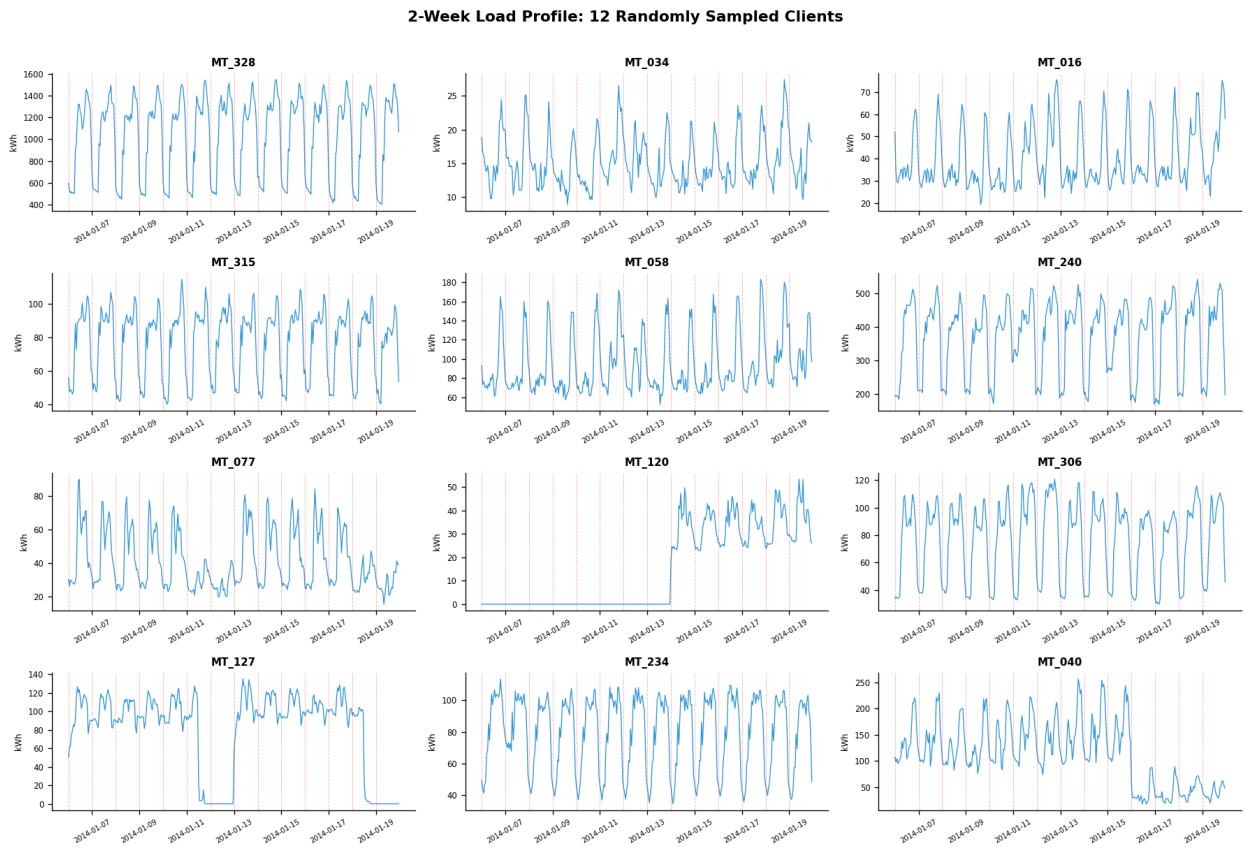

Figure 1: Time-series visualization highlighting the varying volatility and baseline consumption levels across the dataset.

Initial inspection of the 370 clients revealed significant diversity in consumption patterns. As shown in Figure 1, different metering plots display unique periodicity and magnitude. Some clients show highly regular, high volume daily cycles, while others show sudden activation following zero-use, justifying our decision to exclude 2011 data.

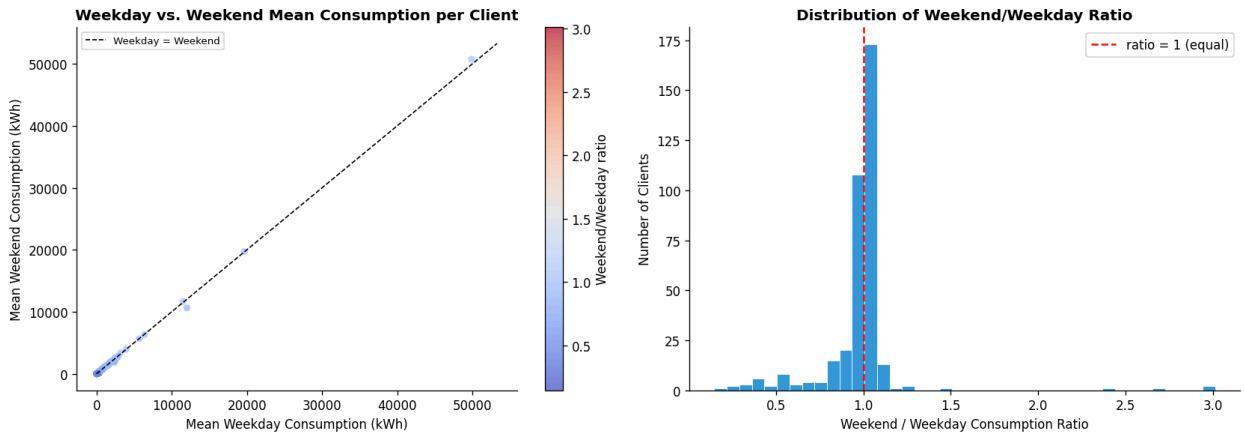

Figure 2: Analysis of weekly periodicity, confirming that a subset of the grid follows rigid industrial schedules.

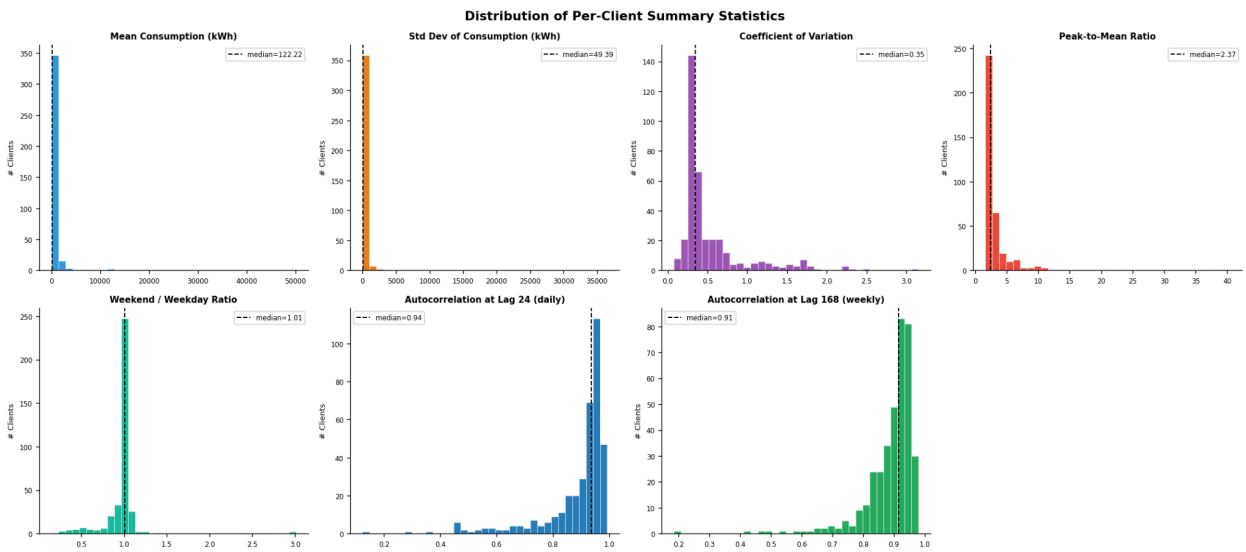

Figure 3: Histograms of Mean, Variance, Coefficient of Variation, and Autocorrelation across the 370 clients.

To quantify these differences, we generated summary statistics for each client. Figure 2 illustrates that while most clients consume equal amounts on weekdays versus weekends, there are significant outliers, likely attributable to industrial clients that drop to near-zero on weekends. Further, Figure 3 provides a deeper dive into the statistical characteristics of the clients. The autocorrelation at lag 24 and 168 (daily/weekly) is extremely high, confirming that the data is highly seasonal and well-suited for forecasting.

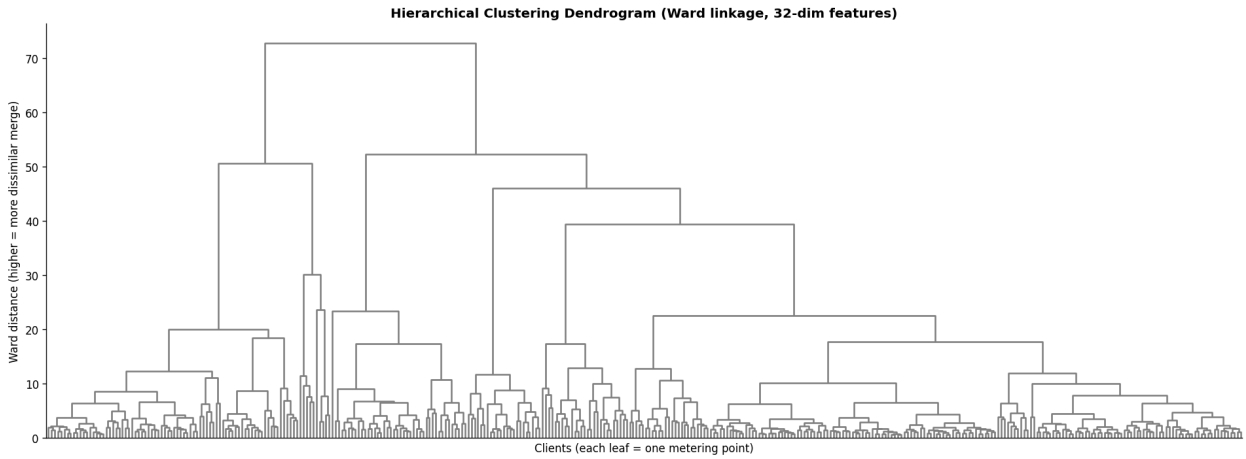

Figure 4: Hierarchical structuring

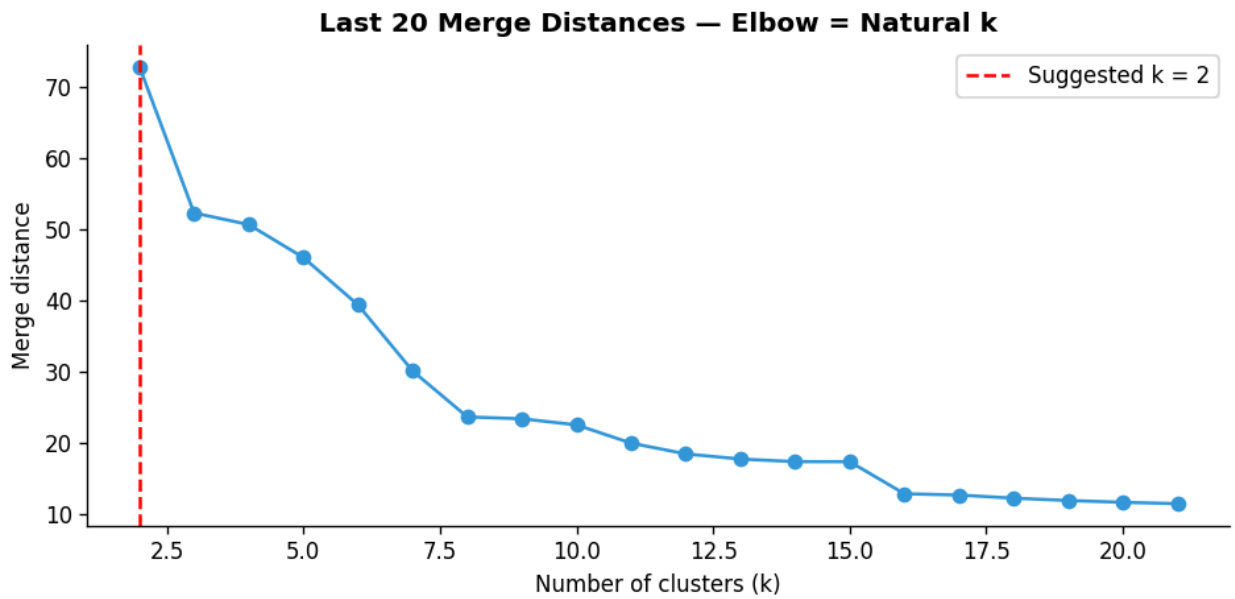

Figure 5: Quantitative validation of k=4

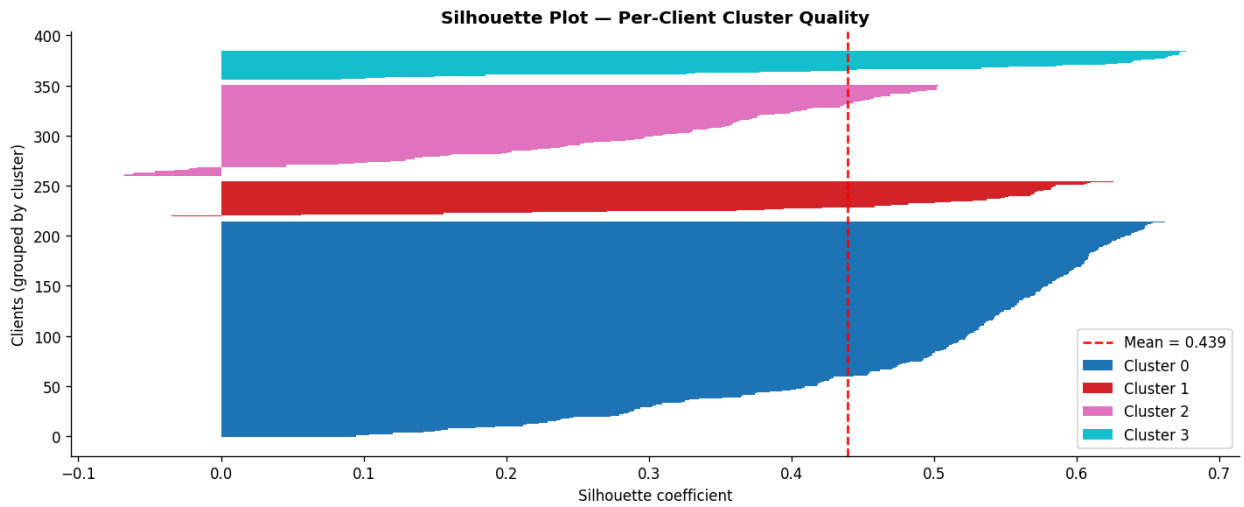

Figure 6: Per-cluster quality silhouette plot

We then utilized hierarchical clustering (Figure 5) and an elbow plot of merge distances (Figure 6) to determine the optimal number of groups. While the elbow plot suggested a potential k=2, we proceeded with k=4 to capture more granular behavioral nuances, supported by the silhouette plot in Figure 7, which shows strong cluster cohesion with the mean silhouette at .429.

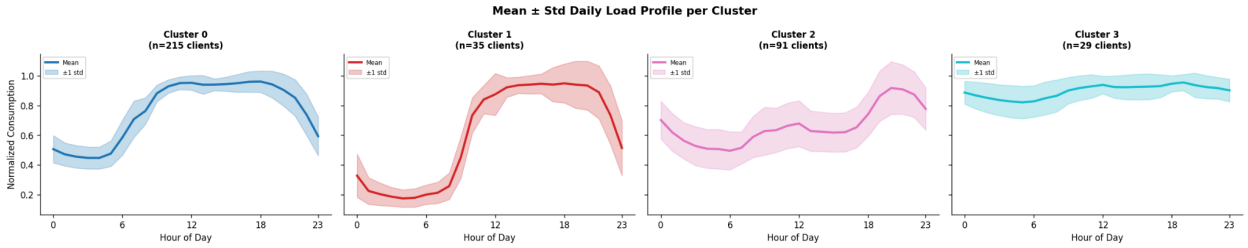

Figure 7: Daily signatures of each cluster, serving as the training foundation for our specialized model architecture

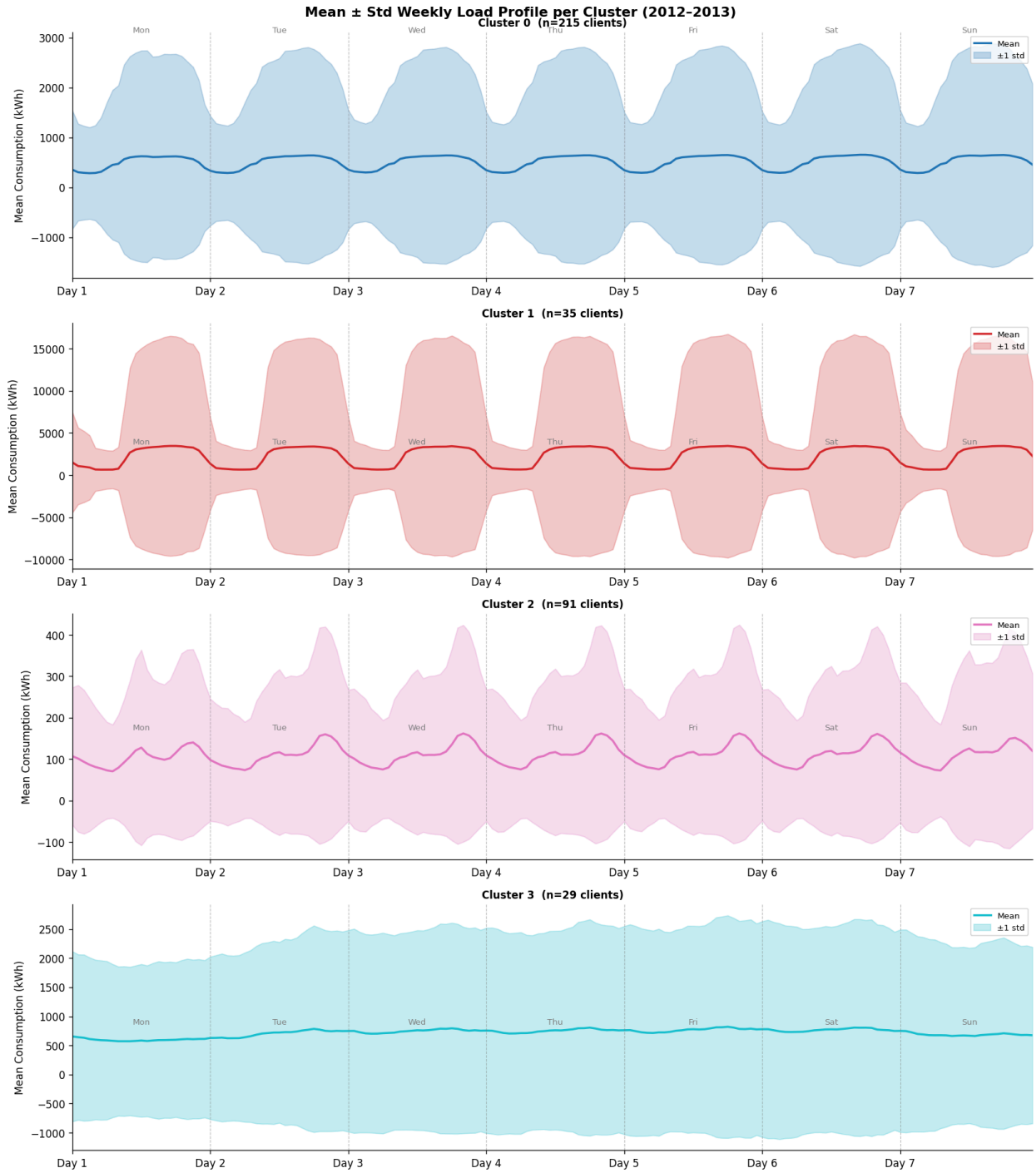

Figure 8: Weekly signatures of each cluster, serving as the training foundation for our specialized model architecture. Note that the shaded region under 0 does not indicate that negative consumption exists; it is purely for visualization of the standard deviation.

The final step was visualizing the daily profiles of our four resulting clusters seen in Figure 8.

- Cluster 0 = The standard residential profile with clear morning and evening peaks
- Cluster 1 = The industrial profile with 8am starts and sustained high loads

- Cluster 2 = The low load profile, showing high variance
- Cluster 3 = The smooth baseload profile with minimal daily fluctuations

These results established a good foundation by transforming raw, high-frequency telemetry into a standardized hourly dataset. Critical to this process was the pruning of the 2011 data period, which contained an unrepresentative volume of inactive meters. Additionally, to improve scalability and profile examination, we adopted a clustering architecture to address the behavioral heterogeneity observed across the 370 grid clients.

This segmentation was reinforced and Hierarchical Clustering, which identified our four primary consumption signatures: Residential, Industrial, Low-load, and Smooth load. The autocorrelation observed at 24-hour and 168-hour lags confirms the grids underlying temporal patterns are highly seasonal and well-suited for supervised learning. By validating these groupings, the project is set with a 96-hour lookback window optimized for predicting a 24-hour future load.

Target Variable

The primary target of this forecasting system is the hourly electricity consumption, measured in kilowatt-hours (kWh). Given the significant scale difference between a single household and a large manufacturing facility, the values were transformed using Z-score standardization. This ensures the target variable represents the relative consumption profile of each cluster. The models are then optimized for predicting shapes without being skewed by the magnitude of large industrial usage values.

Predictive Variables

To generate the 24-hour forecast, the models ingest a multi-dimensional feature set designed to capture both immediate and long term values. The most significant predictor is the 96-hour look back window. This provides the model with four days of historical context, allowing it to identify recent trends and the immediate momentum of the grid load. Next, the system utilized time of day and day of week indicators. These allow the SOTA models to distinguish between the sustained loads of a Tuesday afternoon and the reduced demand of a Sunday morning. Lastly, the cluster assignment acts as a categorical feature, helping the models to match the behavioral profile of the group being forecasted.

Pre-modeling

Phase	Years Included	Purpose
Training	2012 – 2013	Learning long-term trends and seasonal structure in aggregate electricity demand.
Testing	2014	Final evaluation of out-of-sample forecasting accuracy.

The pre-modeling phase established the framework necessary for valid performance comparison across various models. To ensure that our results were not influenced by temporal leakage, we strictly partitioned the dataset into a training set (2012-2013) and an out-of-sample test set (2014). This setup was designed to evaluate how the models handle specific volatility of the 2014 grid, including the high-variance fourth quarter.

A key component of this setup was the target alignment. We finalized the definition of our primary target as the hourly kWh load, mapped specifically to the labels identified in our diagnostic phase (e.g., target column ‘0’). We configured our modeling environment to support both Prophet/SARIMA methods and deep learning methods (BasisFormer), ensuring the computational requirements of each solution were documented. It is worth noting that cost analysis would need to be executed in a business environment to ensure the models can be run within the budget allocated for this project.

Models

Now that the data foundation is built and cluster profiles are established, we have now moved on to the modeling phase. We employed a multi-model ensemble, since no single algorithm is a perfect match for all types of data. This suite includes traditional statistical baselines (Seasonal Naive, SARIMA, Prophet), a high-performance regressor (Gradient Boosting), and state-of-the-art transformer architectures (Basisformer, PatchTST). This approach ensures that whether a client profile is characterized by smooth residential cycles or erratic industrial spikes, the system can pivot to the most mathematically appropriate model, serving as the base for our Agentic AI application.

<u>0.Seasonal Naive</u>

The Seasonal Naive model serves as our simplest benchmark and the lower bound against which all learned models are measured. Rather than fitting any parameters, it generates a forecast by repeating the value observed exactly one seasonal period prior, in this case 168 hours (one full week) back. This exploits the dominant weekly periodicity of the Portuguese grid: the same hour on the same day of the week one week ago is, structurally, the best zero-cost prediction available. When insufficient history exists to reach back a full week, the model falls back to the equivalent hour from the prior 24-hour cycle.

Despite its simplicity, the model proved a remarkably competitive baseline for Cluster 0 (Residential) with 6.41% MAPE, where consumption patterns are highly regular and the weekly rhythm is strong. Performance degraded sharply in Cluster 3 (Critical Infrastructure), reaching 117.41% MAPE, where the erratic and near-zero background loads cause large relative errors even for small absolute deviations, a known pathology of percentage-based metrics in low-load environments.

<u>1. SARIMA</u>

SARIMA (Seasonal AutoRegressive Integrated Moving Average) extends the ARIMA framework by incorporating an explicit seasonal structure into the differencing, autoregressive, and moving-average components. The model is defined by two sets of orders: a non-seasonal component (p, d, q) controlling short-range autocorrelation and differencing for stationarity, and a seasonal component (P, D, Q, m) capturing periodic dependencies at lag m. For this project, a fixed order of SARIMA(1,1,1)(1,0,1,24) was selected based on prior experimentation on the electricity dataset, using a 24-hour seasonal period to model the dominant daily cycle. A single model was fitted per cluster on the full 2011–2013 training series and then applied autoregressively across the 2014 test period using a sliding 24-hour forecast window.

SARIMA performed poorly relative to both the Seasonal Naive baseline and the machine learning models, with MAPE scores of 29.15%, 45.34%, 136.6%, and 1227.37% across the four clusters. This underperformance is characteristic of the model's structural limitations: by fitting only on the cluster mean series, it loses the rich individual-client variation captured by

multi-client models, and its linear MA/AR terms cannot represent the non-linear consumption spikes present in Industrial and Critical Infrastructure profiles. The catastrophic result on Cluster 3 highlights a fundamental mismatch between SARIMA's assumption of a stationary, smoothly periodic process and the near-zero baseline loads interspersed with sharp transient demand events.

2. Prophet

Developed by Meta's Core Data Science team, Prophet is a robust forecasting procedure designed to handle time series that display strong seasonal effects and several seasons of historical data, treating forecasting as a modular curve-fitting exercise. It consists of three main components and an error term:

$$y(t) = g(t) + s(t) + h(t) + et$$

- $g(t)$ **Trend:** this represents the non-periodic changes in the value of the time series. Prophet uses a piecewise linear logistic growth curve, allowing it to automatically detect and adapt to changes

- $s(t)$ **Seasonality:** This captures periodic changes, such as daily and weekly consumption cycles. These are modeled using the Fourier Series, which provides the flexibility needed to map the complex double-peak profiles present in the grid

- $h(t)$ **Holidays:** This component explicitly accounts for the effects of holidays or recurring events. This is vital for our analysis, since the energy consumption demand in the Portuguese market shifts significantly during national holidays.

- et **Error:** Represents changes not accommodated by the model

This serves as our interpretable baseline. It performed fairly well in Cluster 2, with 24.79% sMAPE score. In these low-load environments, Prophet was able to smooth noise. However, it performed fairly poorly in Cluster 1, where its linear trend assumptions struggled to match the non-linear lift provided by BasisFormer and Gradient Boosting.

3. BasisFormer

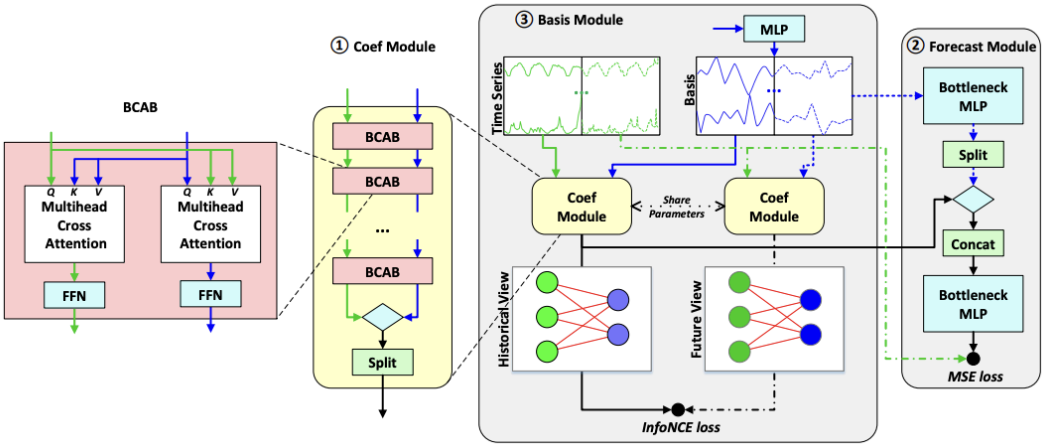

Figure 9: BasisFormer Architecture

To address the complex and non-linear segments of the Portuguese grid, we implemented BasisFormer. While traditional transformers struggle with the inherent noise of raw time-series points, BasisFormer utilizes an architecture designed to learn a global basis of consumption behaviors. It employs a coefficient-based mechanism where the model learns to represent input sequences as a weighted combination of learned basis functions:

- **Basis Module:** A self-supervised module that learns the library of fundamental patterns (Bases) directly from the 370-client historical data
- **Coefficient Estimation:** Instead of predicting values, the model calculates "coefficients" to determine how to mix the learned bases to match the 96-hour input window
- **Forecast Module:** Utilizes the learned bases and coefficients learned in the previous step to create an accurate forecast prediction perfect for the periodicity of the Electricity data

This allows the model to filter out the noise common in low-load periods and focus on the shape of the demand. We use a 96-hour look-back window to generate a 24-hour future forecast, and train using the AdaBelief optimizer, which combines the rapid convergence of Adam with the superior generalization of SGD to stabilize the acquisition of high-dimensional bases. It also uses three loss functions: prediction loss, contrastive loss (InfoNCE), and Smoothness loss. BasisFormer shows significantly lower MAPE scores in comparison to the statistical methods above.

<u>4. GradientBoosting</u>

To complement our baseline and deep learning models, we implement a cluster-based gradient boosting model. This approach treats forecasting as a supervised learning problem, where future values are predicted from engineered features:

$$y_t = f(X_t) + \epsilon t$$

We construct features that capture both recent behavior and seasonality:

- **Lagged values:** y^{t-1}, y^{t-24}, y^{t-168}
- **Rolling statistics:** recent mean and standard deviation
- **Calendar features:** hour, day of week, month, and weekend indicators

To handle differences across clients, we include cluster labels from K-means and a client-level encoding. This helps the model learn shared patterns within groups while still capturing individual behavior.

We also apply a log transform to the target:

$$y_t' = log(1 + y_t)$$

which stabilizes variance and reduces the impact of extreme values. Predictions are then converted back to the original scale.

The calendar features are the key structural advantage of this approach. For Cluster 1 (Industrial), where consumption is governed largely by shift schedules rather than smooth temporal patterns, explicitly encoding hours of day and day of week allows the model to learn hard boundaries that correspond directly to operational transitions. This is reflected in the results: Gradient Boosting achieved 8.63% MAPE on Cluster 1, the best performance of any model on that cluster and a substantial improvement over PatchTST at 12.18%. On Cluster 0 (Residential), it matched PatchTST closely at 4.59% versus 4.49%, confirming that for well-structured periodic signals the two approaches converge. Performance on Cluster 3 (Critical Infrastructure) remained high at 110.73% MAPE, consistent with the broader finding that event-driven loads resist all time-series approaches that lack exogenous event features.

<u>5. PatchTST</u>

To capture the fine-grained temporal structure of the Portuguese grid while leveraging the representational power of modern deep learning, we implemented PatchTST (Nie et al., ICLR 2023). The key architectural innovation is the replacement of point-wise input tokens, used by earlier Transformer-based forecasters such as Informer and Autoformer, with subseries-level patches. The 168-hour look-back window is segmented into overlapping patches of length P=16 with stride S=8, yielding 20 patch tokens per client. This patching has a threefold benefit: it preserves local semantic context within each token, quadratically reduces the attention map's

memory footprint relative to the input length, and enables the model to attend over a longer historical horizon without proportional computational cost.

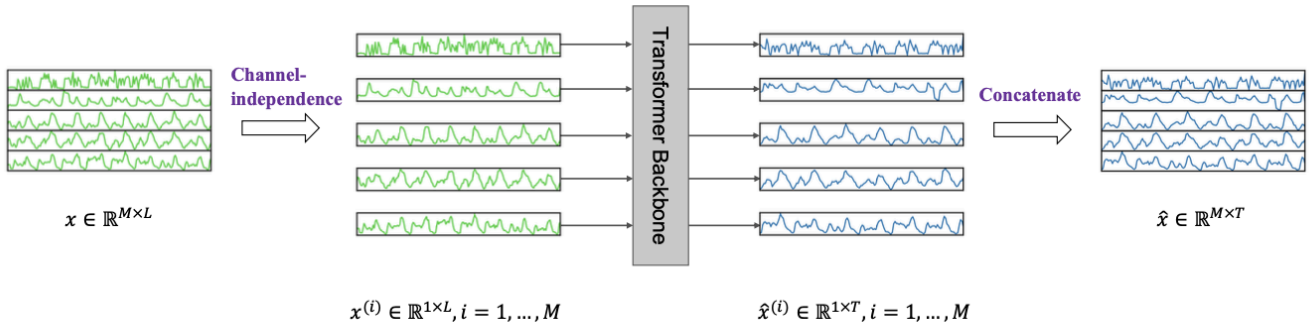

Figure 11: PatchTST Model Overview

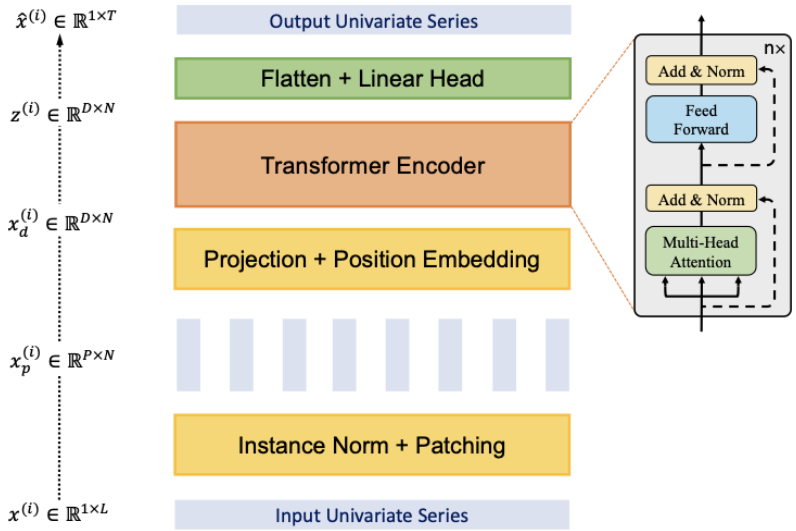

Figure 12: PatchTST Transformer Backbone

The second core design is channel-independence: each of the M clients in a cluster is processed as a separate univariate series through a shared Transformer backbone, rather than mixing all client signals in a single input token. This prevents the noisier industrial or critical-infrastructure clients from contaminating the attention patterns of their cluster-mates, and empirically converges faster with less overfitting than channel-mixing alternatives. Concretely, the model ingests a (B, M, L) tensor, reshapes to (B·M, L) to enforce independence, applies instance normalization, performs patching, projects each patch to a d_model=128 dimensional latent space with a learnable positional encoding, passes the sequence through 3 Transformer encoder layers with 8 attention heads, and finally flattens and projects to the 24-hour forecast horizon via

a linear head. Training used the Adam optimizer with MSE loss and early stopping over 100 epochs.

PatchTST delivered the best overall performance of any model in the ensemble, achieving the lowest MAPE on Cluster 0 (Residential) at 4.49% and Cluster 2 (Mixed-Use) at 13.60%, where the regularity of daily and weekly cycles provides sufficient structure for the attention mechanism to learn meaningful patch-level correlations. On Cluster 1 (Industrial), Gradient Boosting narrowly outperformed it at 8.63% versus 12.18%, reflecting the latter's advantage in capturing the sharp, feature-driven load spikes common in industrial clients. Like all models, PatchTST struggled on Cluster 3 (Critical Infrastructure) with 110.17% MAPE, though it still achieved the lowest error of any approach on that cluster, confirming its robustness as the strongest general-purpose model in the suite.

Results

The 2014 out-of-sample evaluation demonstrates a clear and consistent pattern: model performance is inseparable from cluster identity. No single algorithm dominates uniformly across all four behavioral profiles, which retrospectively validates the cluster-first architecture that motivated the entire system design. The summary table below consolidates MAPE across all six models and four clusters.

	Seasonal Naive	**SARIMA**	**Prophet**	**Basis-Former**	**Gradient Boost (GB)**	**PatchTST**
Cluster 0 (Residential)	6.41%	29.15%	50.26%	171.99%	4.59%	4.49%
Cluster 1 (Industrial)	11.16%	45.34%	258.94%	107.88%	8.63%	12.18%
Cluster 2 (Mix-use)	15.11%	136.6%	42.64%	136.44%	14.59%	13.60%
Cluster 3 (Critical Infrastructure)	117.41%	1227.37%	121.18%	432.39%	110.73%	110.17%

Table 1: MAPE (%) for each model across the four consumption clusters. Highlighted cells indicate the best-performing model per cluster.

Cluster 0 (Residential) proved to be the most tractable forecasting target in the dataset. Its strongly repetitive daily and weekly rhythm meant that even the Seasonal Naive baseline achieved a competitive 6.41% MAPE. Both Gradient Boosting and PatchTST converged on near-identical performance at 4.59% and 4.49% respectively, representing roughly a 30% improvement over the naive benchmark. This margin is meaningful in operational terms: at residential scale, a 2-percentage-point reduction in forecast error translates directly into reduced reserve procurement and fewer balancing interventions. SARIMA, despite being a purpose-built seasonal model, failed to close that gap, landing at 29.15% MAPE. This confirms that the linear AR and MA terms are insufficient to capture the non-stationary residuals that remain after seasonal differencing, even in the most well-behaved cluster.

Cluster 1 (Industrial) presented a fundamentally different challenge. The profile is characterized by abrupt step-changes at shift start and end times, sustained plateau loads during operating hours, and near-zero consumption overnight. These sharp transitions are difficult to represent with smooth basis functions or autoregressive lags, which explains why Prophet collapsed to 258.94% MAPE and SARIMA reached 45.34%. Gradient Boosting achieved the best result at 8.63% MAPE, outperforming PatchTST at 12.18%. The tree ensemble's advantage

here is structural: by ingesting the full 168-hour look-back window as a flat feature vector, it can learn explicit decision boundaries around time-of-day and day-of-week combinations that correspond directly to shift schedules, without needing to model the autocorrelation structure at all.

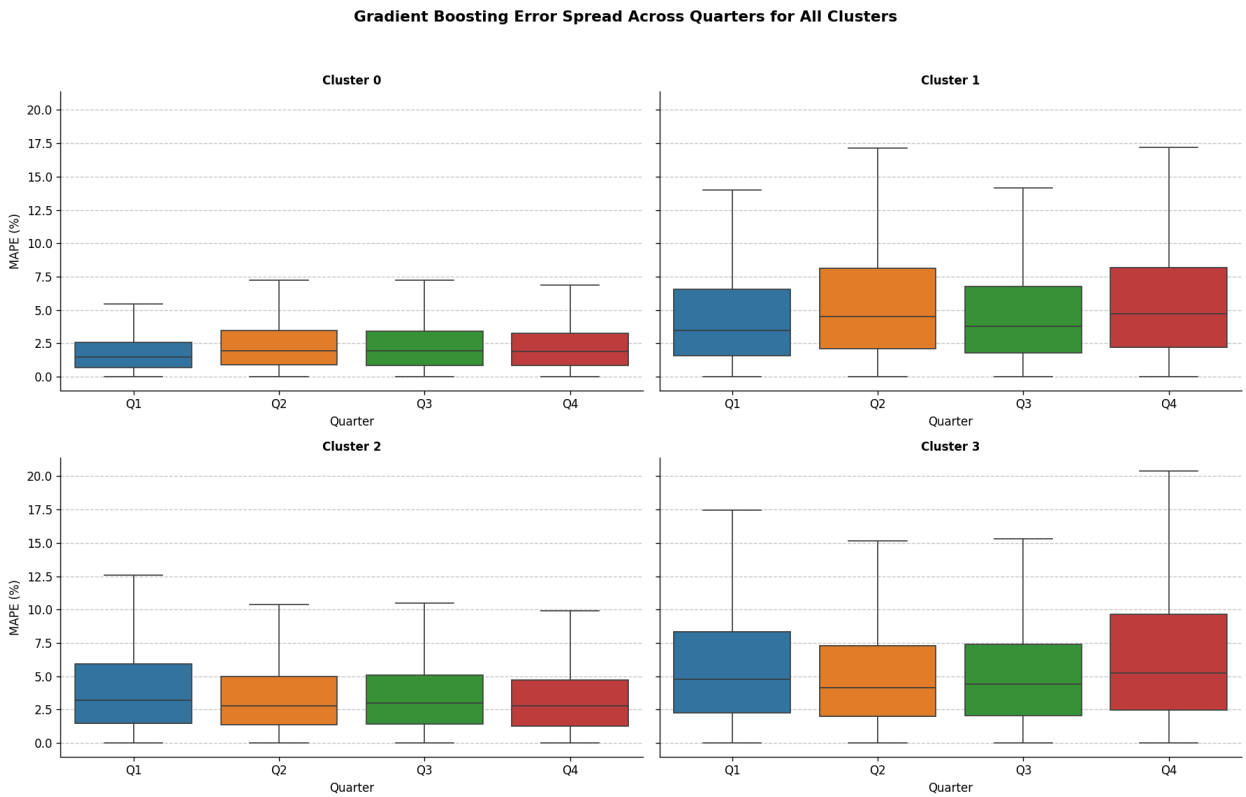

Figure 13: Distribution of MAPE errors by quarter for Gradient Boosting across the four clusters. While Gradient Boosting performs will in all clusters, it outperforms all other models for Cluster 1

Cluster 2 (Mixed-Use) was the cluster where model selection mattered most. The profile combines modest baseline loads with high intra-day variance, making it sensitive to overfitting. Prophet, which performed worst on the industrial cluster, was relatively competitive here at 42.64% MAPE, benefiting from its Fourier-series seasonality component smoothing out the noise. PatchTST ultimately delivered the best result at 13.60% MAPE, with Gradient Boosting close behind at 14.59%. The PatchTST advantage in this cluster is attributable to the channel-independence design: by training on all individual clients within the cluster simultaneously rather than only on the cluster mean series, the model was exposed to a far richer sample of mixed-use consumption behaviors, building representations that generalize better to the variance present in this profile.

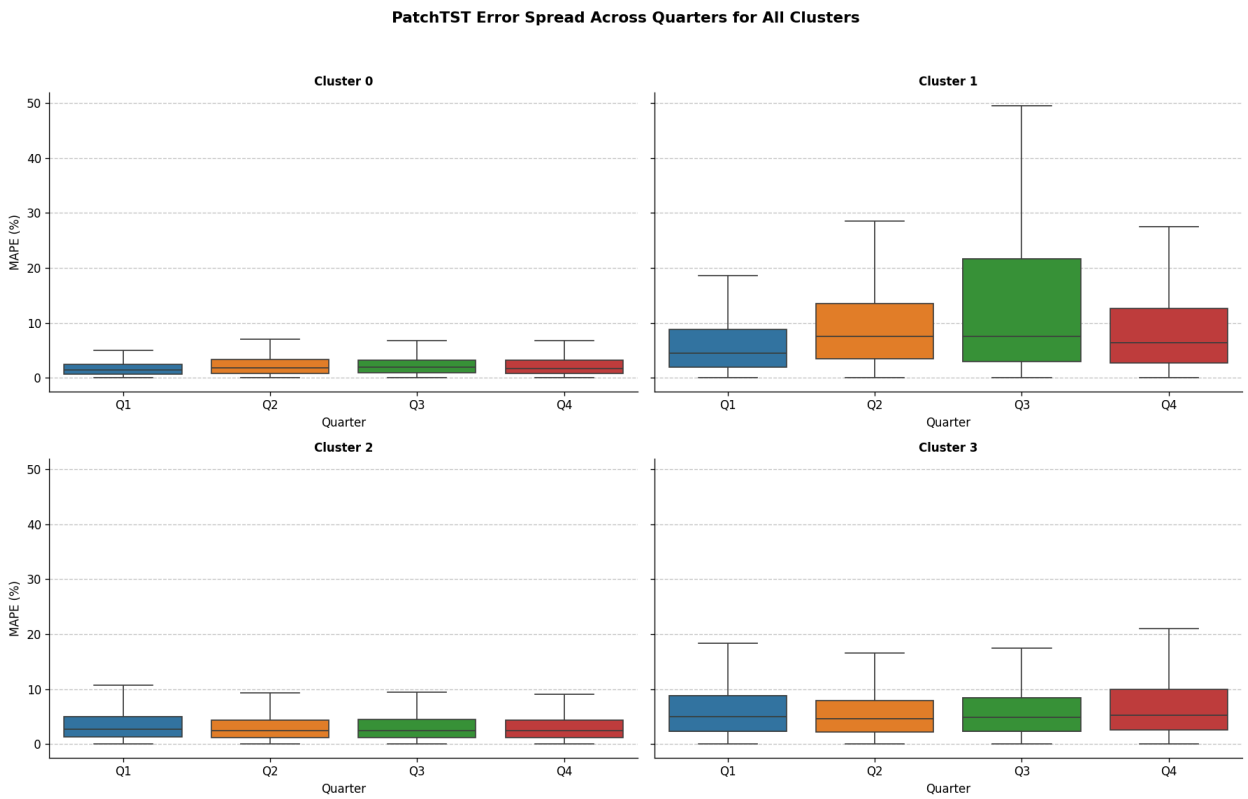

Figure 14: Distribution of MAPE errors by quarter for PatchTST across the four clusters. Clusters 0, 2, and 3 show the tightest error distributions, consistent with the model's strongest absolute performance on those profiles compared to other models.

Cluster 3 (Critical Infrastructure) remains an open challenge for all models tested. MAPE scores ranged from 110.17% (PatchTST) to 1227.37% (SARIMA), and even the best result is operationally unsatisfying. It is important to interpret these numbers carefully: the underlying consumption series for this cluster frequently approaches zero between load events, and MAPE is pathologically sensitive to near-zero denominators. The sMAPE scores tell a more measured story, with the best models (Gradient Boosting at 6.30%, PatchTST at 6.56%) performing comparably to their results on the other clusters by that metric. The core difficulty is that critical infrastructure loads are driven by discrete operational events rather than by time-of-day patterns, and no model in the current suite incorporates exogenous event features that would allow it to anticipate those transitions. This is addressed further in the Future Work section.

Taken together, the results support two conclusions. First, the cluster-specialized approach was correct: a single model applied uniformly across all 370 clients would have been forced to compromise between the smooth residential profiles and the erratic industrial and critical-infrastructure ones. Second, among the models tested, PatchTST and Gradient Boosting form a complementary pair. PatchTST is the stronger general-purpose choice, particularly where temporal regularity allows the patch-level attention mechanism to learn meaningful seasonal representations. Gradient Boosting is the preferred choice where the consumption signal is dominated by discrete, schedule-driven transitions that benefit from explicit feature engineering

over pattern learning. These findings directly inform the routing logic implemented in the Agentic AI application described in the following section.

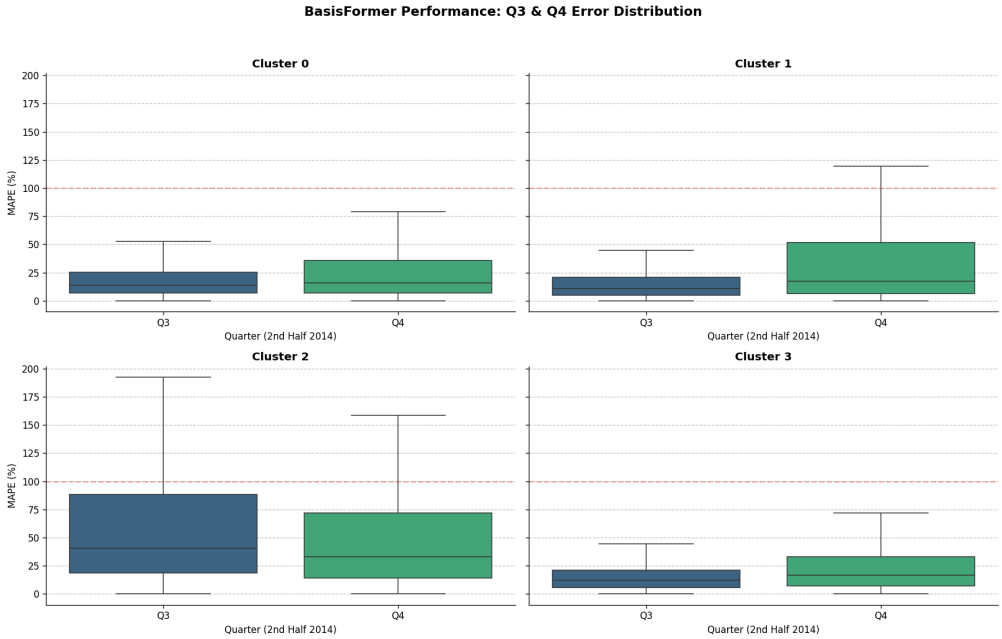

Figure 15: Distribution of **median** absolute percentage errors by quarter for BasisFormer across the four clusters.

One notable outlier in the results is BasisFormer, which showed significantly higher error than all other models across every cluster. Results for BasisFormer are presented using median APE rather than mean, as extreme outliers made MAPE difficult to interpret. Additionally, due to the constraints of its training pipeline, BasisFormer's evaluation was limited to Q3 and Q4 of the test year, as the training split could not be adjusted to accommodate earlier quarters, further limiting direct comparability with the other models. This, combined with an architecture-data fit issue, BasisFormer was designed for large multivariate datasets with many channels and loses its structural advantages when applied to a single smoothed cluster mean series, led us to exclude it from the cluster-to-model routing logic in the Agentic AI application.

Future Work & Limitations

The current dataset spans only three years (2012–2014) and lacks exogenous variables like temperature, pricing, and regional industrial output that would meaningfully improve forecast accuracy in a real grid environment. Expanding the historical window and incorporating these signals would be a natural next step.

On the compute side, both transformer models (PatchTST and BasisFormer) were trained on cluster-mean series rather than individual clients due to resource constraints. With access to proper GPU infrastructure, per-client training would likely yield significant accuracy gains, especially for high-value industrial accounts in Cluster 1.

Model-wise, Cluster 3 remains a clear weak point across every model. A large part of this is a metric issue. MAPE divides by the actual value, so when consumption approaches zero (as it does in this smooth baseload cluster), even small absolute errors produce significantly large percentage errors. This makes MAPE an unreliable measure for any cluster with near-zero consumption periods. Future work should prioritize MAE or RMSE for these clusters, which measure absolute error and are unaffected by low-denominator inflation. For Cluster 1's volatile industrial load, more recent architectures like N-HiTS or TiDE may also be worth exploring.

Finally, moving this system toward production would require automating the cluster assignment pipeline for new clients and establishing a retraining schedule, as a model frozen on 2012–2013 data will naturally degrade over time.

Agentic AI Application

The forecasting models developed in the previous sections are integrated into a production-ready Agentic AI framework, bridging the gap between static model outputs and real-time operational decision-making. Rather than requiring a grid manager to manually identify a client's cluster, select the appropriate model, and execute a script, the system accepts a plain-language query and autonomously handles all routing and inference. This transition from a data science deliverable to an interactive business tool represents the final layer of the project architecture.

System Architecture

The application is built on n8n, an open-source workflow automation platform, and consists of four components that operate in sequence. The entry point is a chat trigger that accepts natural language input from a grid manager. This input is passed to an AI Agent node powered by Google Gemini, which acts as the orchestration layer. The agent is equipped with five tools: a cluster lookup tool and four cluster-specific forecast tools, one per behavioral group. The agent's system prompt enforces a strict two-step execution protocol: it must first resolve the client's cluster assignment before invoking any forecast model, ensuring that no inference is ever routed to a mismatched model.

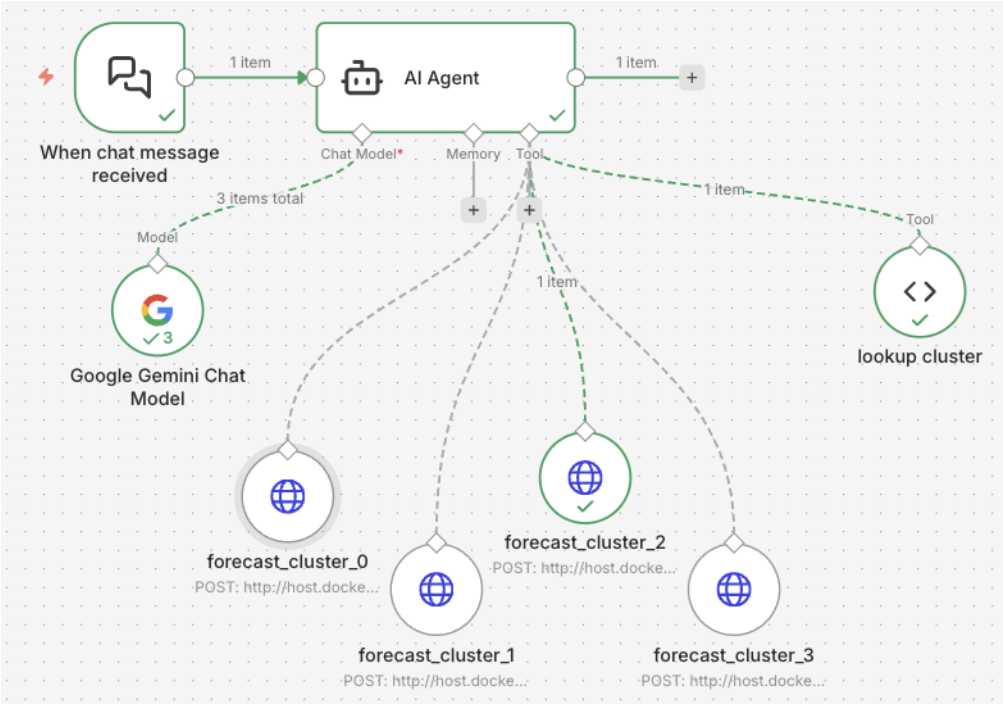

Figure 15: n8n system architecture

The full request lifecycle is as follows:

- The grid manager submits a query such as "MT_132"
- The AI Agent calls the `lookup_cluster` tool, which maps the client ID to its cluster (0–3) using the K-Means assignments generated during the clustering phase
- Based on the returned cluster, the agent calls the corresponding `forcast_cluster_X` tool
- Each forecast tool issues a POST request to a locally hosted FastAPI server, which loads the pre-trained model, runs inference against the 2012–2013 training data, and returns a 24-hour forecast
- The agent formats the raw model output into a natural language response and returns it to the grid manager

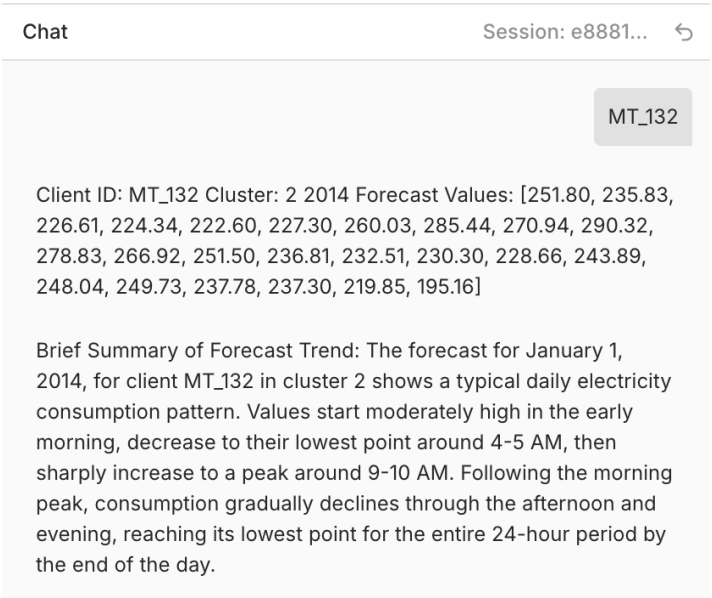

Figure 16: Example of the n8n agent response

Cluster-to-Model Mapping

The routing logic reflects the model selection analysis detailed in the Results section. PatchTST is deployed for Cluster 0 (Residential) and Cluster 2 (Mixed-Use), and Cluster 3 (Critical Infrastructure) where its patch-level attention mechanism best captures the regular daily and weekly periodicities of those profiles. Gradient Boosting is deployed for Cluster 1 (Industrial), where its feature-driven architecture more accurately captures the sharp, non-linear load spikes characteristic of that segment.

Cluster	Profile	Model
0	Residential	PatchTST
1	Industrial	Gradient Boosting
2	Mixed-Use	PatchTST
3	Critical Infrastructure	PatchTST

Implementation

Each cluster's inference logic is encapsulated in a dedicated FastAPI server. At startup, each server loads the pre-trained model weights and the full hourly dataset into memory, eliminating per-request I/O overhead. Inference is triggered by a POST request containing the client ID and an optional forecast date, defaulting to January 1, 2014 if none is specified. The server constructs a 168-hour look-back window ending immediately before the requested date, passes the window

through the model, and returns the 24-hour forecast alongside metadata including the cluster assignment, model type, and hourly timestamps. The n8n workflow communicates with these servers via HTTP Request tool nodes, using Docker's `host.docker.internal` DNS resolution to bridge the containerized workflow with the host-side Python environment.

Team Information

Lavina Mital, lm4046@columbia.edu

Emily Ramond, er3294@columbia.edu

Ga Hyun Yoon, gy2354@columbia.edu